JAVA WEB

Tutorial da Aula 03

Persistência com PostgreSQL e JDBC

Continuação do projeto PrimeiraAplicacaoWeb
NetBeans + Ant + JDK 17 ou superior + Tomcat 11 + PostgreSQL
Sem Maven

Objetivo: substituir o armazenamento em memória da Aula 02 por persistência real em banco de dados, utilizando JDBC e o padrão DAO.

1. O que será construído

A aplicação continuará oferecendo listar, cadastrar, editar, excluir e validar alunos. A diferença é que os dados passarão a ser armazenados permanentemente no PostgreSQL.

```
Fluxo anterior:
Navegador -> AlunoServlet -> AlunoRepositorio -> Map em memória

Novo fluxo:
Navegador -> AlunoServlet -> AlunoDAO -> JDBC -> PostgreSQL
```

Objetivos da Aula 03

- Compreender o papel de um banco de dados em uma aplicação web.
- Conectar Java ao PostgreSQL utilizando JDBC.
- Adicionar manualmente o driver JDBC em um projeto Ant.
- Criar uma classe de conexão.
- Executar comandos SQL através de Java.
- Utilizar PreparedStatement e ResultSet.
- Implementar CRUD utilizando DAO.
- Fechar corretamente recursos JDBC.
- Tratar erros de banco de dados.
- Integrar Servlet, DAO, modelo e JSP.

2. Arquitetura da aplicação

Na Aula 02, o sistema possuía modelo, repositório em memória, Servlet e páginas JSP. Nesta etapa, o repositório temporário será substituído por uma camada DAO e uma classe de conexão.

```
br.edu.exemplo.modelo
  Aluno.java

br.edu.exemplo.dao
  AlunoDAO.java

br.edu.exemplo.conexao
  Conexao.java

br.edu.exemplo.controle
  AlunoServlet.java

WEB-INF/views
  lista.jsp
  formulario.jsp
```

3. O que é JDBC?

JDBC significa Java Database Connectivity. É a API utilizada pelo Java para comunicação com bancos de dados relacionais.

```
Java -> JDBC -> Driver PostgreSQL -> PostgreSQL
```

Elemento	Responsabilidade
JDBC	API padronizada para acesso a bancos relacionais.
Driver PostgreSQL	Implementação que permite ao Java conversar com o PostgreSQL.
Connection	Representa uma conexão aberta com o banco.
PreparedStatement	Executa comandos SQL parametrizados.
ResultSet	Representa resultados retornados por consultas SELECT.

4. Preparar o PostgreSQL

Crie o banco de dados:

```
CREATE DATABASE primeira_aplicacao_web;
```

Conecte-se ao banco criado e execute a criação da tabela:

```
CREATE TABLE aluno (
    id SERIAL PRIMARY KEY,
    nome VARCHAR(120) NOT NULL,
    email VARCHAR(150) NOT NULL,
    curso VARCHAR(120) NOT NULL,
    semestre INTEGER NOT NULL,
    turno VARCHAR(30) NOT NULL
);
```

Campo	Tipo	Finalidade
id	SERIAL	Identificador automático
nome	VARCHAR(120)	Nome do aluno
email	VARCHAR(150)	E-mail
curso	VARCHAR(120)	Curso
semestre	INTEGER	Semestre atual
turno	VARCHAR(30)	Turno

5. Inserir dados iniciais

```
INSERT INTO aluno
(nome, email, curso, semestre, turno)
VALUES
('Ana Silva', 'ana@email.com', 'Sistemas de Informação', 2, 'Noturno');

INSERT INTO aluno
(nome, email, curso, semestre, turno)
VALUES
('Carlos Souza', 'carlos@email.com', 'Análise e Desenvolvimento de Sistemas', 4, 'Noturno');
```

Teste a tabela:

```
SELECT * FROM aluno;
```

6. Adicionar o Driver JDBC ao projeto

Como o projeto continua usando Ant e não Maven, o driver JDBC do PostgreSQL deverá ser adicionado manualmente.

```
NetBeans:
Botão direito no projeto -> Properties -> Libraries -> Add JAR/Folder
```

Selecione o arquivo do driver PostgreSQL, com nome semelhante a postgresql-xx.x.x.jar. Depois, confirme se ele aparece em Libraries.

7. Criar a classe de conexão

Em Source Packages, crie o pacote br.edu.exemplo.conexao e dentro dele a classe Conexao.java.

```
package br.edu.exemplo.conexao;

import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;

public class Conexao {

   private static final String URL =
         "jdbc:postgresql://localhost:5432/primeira_aplicacao_web";
   private static final String USUARIO = "postgres";
   private static final String SENHA = "postgres";

   public static Connection conectar() throws SQLException {
      return DriverManager.getConnection(URL, USUARIO, SENHA);
   }
}
```

A senha deve ser ajustada conforme a instalação local do PostgreSQL.

Parte da URL	Significado
jdbc	API de conexão
postgresql	Banco utilizado
localhost	Servidor do banco
5432	Porta padrão
primeira_aplicacao_web	Nome do banco

8. Testar a conexão

```
package br.edu.exemplo.conexao;

import java.sql.Connection;

public class TesteConexao {
   public static void main(String[] args) {
      try {
```

```
            Connection conexao = Conexao.conectar();
            System.out.println("Conexão realizada com sucesso!");
            conexao.close();
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

Erro	Verificação
Connection refused	Confirme se o PostgreSQL está iniciado e a porta está correta.
password authentication failed	Confira usuário e senha.
No suitable driver	Confirme se o JAR JDBC foi adicionado às Libraries.

9. O que é DAO?

DAO significa Data Access Object. Seu objetivo é concentrar o código responsável por acessar o banco de dados.

```
AlunoServlet -> AlunoDAO -> PostgreSQL
```

Crie o pacote br.edu.exemplo.dao e dentro dele AlunoDAO.java.

10. Estrutura inicial de AlunoDAO

```
package br.edu.exemplo.dao;

import br.edu.exemplo.conexao.Conexao;
import br.edu.exemplo.modelo.Aluno;
import java.sql.Connection;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.util.ArrayList;
import java.util.List;

public class AlunoDAO {

}
```

O DAO deverá fornecer as operações listar(), buscarPorId(), inserir(), atualizar() e excluir().

11. PreparedStatement

PreparedStatement permite executar SQL parametrizado. Em vez de concatenar diretamente valores, utiliza-se o caractere ? como marcador de parâmetro.

```
SELECT * FROM aluno WHERE id = ?

comando.setInt(1, id);
```

12. Método listar()

```
public List<Aluno> listar() throws SQLException {
   List<Aluno> alunos = new ArrayList<>();

   String sql =
      "SELECT id, nome, email, curso, semestre, turno " +
      "FROM aluno ORDER BY id";

   try (Connection conexao = Conexao.conectar();
       PreparedStatement comando = conexao.prepareStatement(sql);
       ResultSet resultado = comando.executeQuery()) {

      while (resultado.next()) {
         Aluno aluno = new Aluno();
         aluno.setId(resultado.getInt("id"));
         aluno.setNome(resultado.getString("nome"));
         aluno.setEmail(resultado.getString("email"));
         aluno.setCurso(resultado.getString("curso"));
         aluno.setSemestre(resultado.getInt("semestre"));
         aluno.setTurno(resultado.getString("turno"));
         alunos.add(aluno);
      }
   }

   return alunos;
}
```

13. Entendendo o ResultSet

executeQuery() devolve um ResultSet. O método next() avança o cursor pelos registros retornados. Cada registro é transformado em um objeto Aluno.

```
resultado.next();
resultado.getInt("id");
resultado.getString("nome");
```

14. Método buscarPorId()

```
public Aluno buscarPorId(int id) throws SQLException {
   String sql =
      "SELECT id, nome, email, curso, semestre, turno " +
      "FROM aluno WHERE id = ?";

   try (Connection conexao = Conexao.conectar();
       PreparedStatement comando = conexao.prepareStatement(sql)) {
```

```
        comando.setInt(1, id);

        try (ResultSet resultado = comando.executeQuery()) {
            if (resultado.next()) {
                Aluno aluno = new Aluno();
                aluno.setId(resultado.getInt("id"));
                aluno.setNome(resultado.getString("nome"));
                aluno.setEmail(resultado.getString("email"));
                aluno.setCurso(resultado.getString("curso"));
                aluno.setSemestre(resultado.getInt("semestre"));
                aluno.setTurno(resultado.getString("turno"));
                return aluno;
            }
        }
    }
    return null;
}
```

15. Método inserir()

```
public void inserir(Aluno aluno) throws SQLException {
    String sql =
        "INSERT INTO aluno " +
        "(nome, email, curso, semestre, turno) " +
        "VALUES (?, ?, ?, ?, ?)";

    try (Connection conexao = Conexao.conectar();
         PreparedStatement comando = conexao.prepareStatement(sql)) {

        comando.setString(1, aluno.getNome());
        comando.setString(2, aluno.getEmail());
        comando.setString(3, aluno.getCurso());
        comando.setInt(4, aluno.getSemestre());
        comando.setString(5, aluno.getTurno());
        comando.executeUpdate();
    }
}
```

16. executeQuery() x executeUpdate()

Método	Uso
executeQuery()	Comandos que retornam registros, principalmente SELECT.
executeUpdate()	Comandos que modificam dados: INSERT, UPDATE e DELETE.

17. Método atualizar()

```
public void atualizar(Aluno aluno) throws SQLException {
    String sql =
        "UPDATE aluno SET " +
        "nome = ?, email = ?, curso = ?, semestre = ?, turno = ? " +
        "WHERE id = ?";

    try (Connection conexao = Conexao.conectar();
         PreparedStatement comando = conexao.prepareStatement(sql)) {

        comando.setString(1, aluno.getNome());
        comando.setString(2, aluno.getEmail());
        comando.setString(3, aluno.getCurso());
        comando.setInt(4, aluno.getSemestre());
        comando.setString(5, aluno.getTurno());
        comando.setInt(6, aluno.getId());
        comando.executeUpdate();
    }
}
```

18. Método excluir()

```
public boolean excluir(int id) throws SQLException {
    String sql = "DELETE FROM aluno WHERE id = ?";

    try (Connection conexao = Conexao.conectar();
         PreparedStatement comando = conexao.prepareStatement(sql)) {
        comando.setInt(1, id);
        return comando.executeUpdate() > 0;
    }
}
```

19. try-with-resources

A estrutura try-with-resources fecha automaticamente Connection, PreparedStatement e ResultSet ao final do bloco, reduzindo o risco de recursos permanecerem abertos.

```
try (Connection conexao = Conexao.conectar();
     PreparedStatement comando = conexao.prepareStatement(sql)) {
    // código
}
```

20. Substituir AlunoRepositorio no Servlet

Troque o repositório em memória pelo DAO.

```
// antes
private final AlunoRepositorio repositorio = new AlunoRepositorio();
```

```
// agora
private final AlunoDAO dao = new AlunoDAO();
```

21. Alterar a listagem

```
private void listar(HttpServletRequest request,
                    HttpServletResponse response)
      throws ServletException, IOException {

   try {
      request.setAttribute("alunos", dao.listar());
      request.getRequestDispatcher("/WEB-INF/views/lista.jsp")
            .forward(request, response);
   } catch (SQLException e) {
      throw new ServletException("Erro ao listar alunos.", e);
   }
}
```

22. Alterar a edição

```
Aluno aluno = dao.buscarPorId(id);
```

O restante da lógica de abertura do formulário pode permanecer semelhante à Aula 02.

23. Alterar o salvamento

```
try {
   if (aluno.getId() == 0) {
      dao.inserir(aluno);
   } else {
      dao.atualizar(aluno);
   }

   response.sendRedirect(
      request.getContextPath() + "/alunos"
   );

} catch (SQLException e) {
   throw new ServletException("Erro ao salvar aluno.", e);
}
```

O sendRedirect permanece porque o sistema continua utilizando o padrão Post/Redirect/Get após um POST válido.

24. Alterar a exclusão

```
private void excluir(HttpServletRequest request,
                     HttpServletResponse response)
      throws IOException, ServletException {
```

```
    try {
        int id = Integer.parseInt(request.getParameter("id"));
        dao.excluir(id);
        response.sendRedirect(request.getContextPath() + "/alunos");
    } catch (SQLException e) {
        throw new ServletException("Erro ao excluir aluno.", e);
    }
}
```

25. As JSPs precisam mudar?

Não necessariamente. lista.jsp continua recebendo o atributo alunos e formulario.jsp continua recebendo aluno. A origem dos objetos muda de memória para PostgreSQL, mas a camada de visualização não precisa conhecer essa diferença.

26. Estrutura final

```
PrimeiraAplicacaoWeb

Source Packages
|- br.edu.exemplo.modelo
|   `- Aluno.java
|- br.edu.exemplo.conexao
|   `- Conexao.java
|- br.edu.exemplo.dao
|   `- AlunoDAO.java
`- br.edu.exemplo.controle
    `- AlunoServlet.java

Web Pages
|- css/estilo.css
|- WEB-INF/views/lista.jsp
|- WEB-INF/views/formulario.jsp
`- index.jsp
```

27. Arquitetura da Aula 03

```
NAVEGADOR
    |
    v
AlunoServlet
    |
    v
AlunoDAO
    |
    v
JDBC
    |
```

```
  v
PostgreSQL
  |
  v
AlunoDAO
  |
  v
AlunoServlet
  |
  v
JSP
  |
  v
NAVEGADOR
```

28. Roteiro de testes

- Iniciar o PostgreSQL.
- Executar Clean and Build no projeto.
- Executar Run Project.
- Abrir http://localhost:8080/PrimeiraAplicacaoWeb/.
- Confirmar a listagem dos alunos existentes.
- Cadastrar um novo aluno e verificar com SELECT * FROM aluno;.
- Editar curso ou semestre e confirmar no banco.
- Excluir um registro e confirmar no banco.
- Reiniciar o Tomcat e verificar que os dados permanecem.

29. Validação continua necessária

A persistência em banco não substitui as validações da aplicação. Devem continuar sendo verificados nome, e-mail, curso, semestre e turno antes de enviar o objeto ao DAO.

30. Responsabilidade de cada camada

Camada	Responsabilidade
Aluno	Representar o aluno.
AlunoServlet	Controlar requisições e fluxo.
AlunoDAO	Executar acesso ao banco.
Conexao	Criar conexões JDBC.
PostgreSQL	Persistir os dados.
JSP	Apresentar a interface.

31. O que o Servlet não deve fazer

Evite colocar SQL diretamente no Servlet. Isso mistura HTTP, validação e persistência na mesma classe. O Servlet deve controlar a requisição e o DAO deve concentrar SQL, conexões e ResultSet.

```
Servlet -> HTTP e fluxo
DAO -> SQL, Connection, PreparedStatement e ResultSet
```

32. Erros comuns

Problema	Verificação
Driver não encontrado	Confira Properties -> Libraries e o JAR do PostgreSQL.
PostgreSQL não conecta	Confira servidor, porta, banco, usuário e senha.
relation aluno does not exist	Confirme se a tabela foi criada no banco correto.
Erro de autenticação	Revise USUARIO e SENHA em Conexao.java.
Dados continuam desaparecendo	Confirme se o Servlet usa AlunoDAO e não AlunoRepositorio.

33. Atividade prática - adicionar RA

Amplie o sistema adicionando o campo RA ao banco e ao modelo. A regra deve impedir valores duplicados.

```
ALTER TABLE aluno
ADD COLUMN ra VARCHAR(30);

ALTER TABLE aluno
ADD CONSTRAINT aluno_ra_unique UNIQUE (ra);
```

```
private String ra;

public String getRa() {
    return ra;
}

public void setRa(String ra) {
    this.ra = ra;
}
```

Atualize INSERT, UPDATE e SELECT no DAO, além do formulário e da tabela JSP.

34. Desafio - busca por nome

Implemente a URL /alunos?busca=ana e realize a busca diretamente no PostgreSQL.

```
SELECT *
FROM aluno
WHERE LOWER(nome) LIKE LOWER(?)
ORDER BY nome;

comando.setString(1, "%" + busca + "%");
```

Crie no DAO:

```
public List<Aluno> buscarPorNome(String nome)
```

35. Comparação das três etapas

Aula	Fluxo principal
Aula 01	HTML -> Servlet -> objeto Pessoa -> JSP

Aula 02	JSP -> Servlet -> AlunoRepositorio -> Map em memória
Aula 03	JSP -> Servlet -> AlunoDAO -> JDBC -> PostgreSQL

36. Síntese da Aula 03

- PostgreSQL e persistência de dados.
- JDBC e driver JDBC.
- Connection e DriverManager.
- PreparedStatement e ResultSet.
- executeQuery() e executeUpdate().
- Padrão DAO.
- try-with-resources.
- CRUD persistente.
- Tratamento de SQLException.
- Integração Servlet -> DAO -> PostgreSQL.
- Manutenção do padrão Post/Redirect/Get.

37. Fluxo final

```
Navegador
 -> GET ou POST
 -> AlunoServlet
 -> Validação
 -> AlunoDAO
 -> PreparedStatement
 -> JDBC
 -> PostgreSQL
 -> ResultSet
 -> AlunoDAO
 -> AlunoServlet
 -> JSP
 -> HTML
 -> Navegador
```

38. Checklist final

- PostgreSQL instalado e funcionando.
- Banco primeira_aplicacao_web criado.
- Tabela aluno criada.
- Driver PostgreSQL adicionado ao projeto Ant.
- Conexao.java funcionando.
- AlunoDAO.java criado.
- Listagem recuperando dados do banco.
- Cadastro persistindo no PostgreSQL.
- Edição funcionando.
- Exclusão funcionando.

- Dados permanecendo após reiniciar o Tomcat.
- Validações da Aula 02 preservadas.
- JSPs em WEB-INF/views.
- POST válido seguido de sendRedirect.
- Clean and Build sem erros.

39. Próxima etapa sugerida

A próxima aula pode evoluir a arquitetura de persistência com DataSource e pool de conexões do Tomcat, transações, tratamento centralizado de erros, filtros e autenticação/sessão.